

# Text Mining 1

Anthony Kamau

## Contents

|                                       |          |
|---------------------------------------|----------|
| <b>Introduction</b>                   | <b>1</b> |
| Load data . . . . .                   | 1        |
| <b>Preparation and Pre-processing</b> | <b>2</b> |
| Visualization . . . . .               | 2        |
| Word cloud . . . . .                  | 2        |
| Frequency plot . . . . .              | 3        |
| Remove stop words . . . . .           | 5        |
| Adjust stopwords list. . . . .        | 6        |
| Stemming . . . . .                    | 7        |
| <b>Regular Expressions</b>            | <b>8</b> |

---

```
# install.packages(c("tidytext","tm", "wordcloud", "SnowballC"))  
  
library(tidyverse)  
library(tidytext)  
library(tm)  
library(wordcloud)  
library(SnowballC)
```

## Introduction

We use the `computer.txt` dataset. It's a data set with reviews of computers. This data set is annotated using aspect-based sentiment analysis. Aspect-based sentiment analysis is a text analysis technique that categorizes data by specific aspects and identifies the sentiment attributed to each one.

## Load data

```
computer_531 <- read_delim(
  file = "computer.txt",
  delim = "\n",
  col_names = "review",
  show_col_types = FALSE
)
```

```
head(computer_531)
```

```
## # A tibble: 6 x 1
##   review
##   <chr>
## 1 ## I purchased this monitor because of budgetary concerns .
## 2 inexpensive[+1][a] ## This item was the most inexpensive 17 inch monitor avai~
## 3 monitor[-1] ## My overall experience with this monitor was very poor .
## 4 screen[-1], picture quality[-1] ## When the screen was n't contracting or gli~
## 5 monitor[-1], picture quality[-1] ## I 've viewed numerous different monitor m~
## 6 screen[-1] ## A week out of the box and I began to see slight contractions of~
```

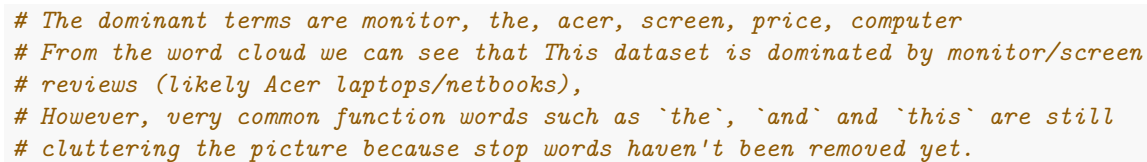
## Preparation and Pre-processing

### Visualization

#### Word cloud

We shall visualize using a word cloud.

```
wordcloud(
  words = computer_531$review,
  min.freq = 5,
  max.words = 50,
  random.order = FALSE,
  colors = brewer.pal(8, "Dark2"))
```



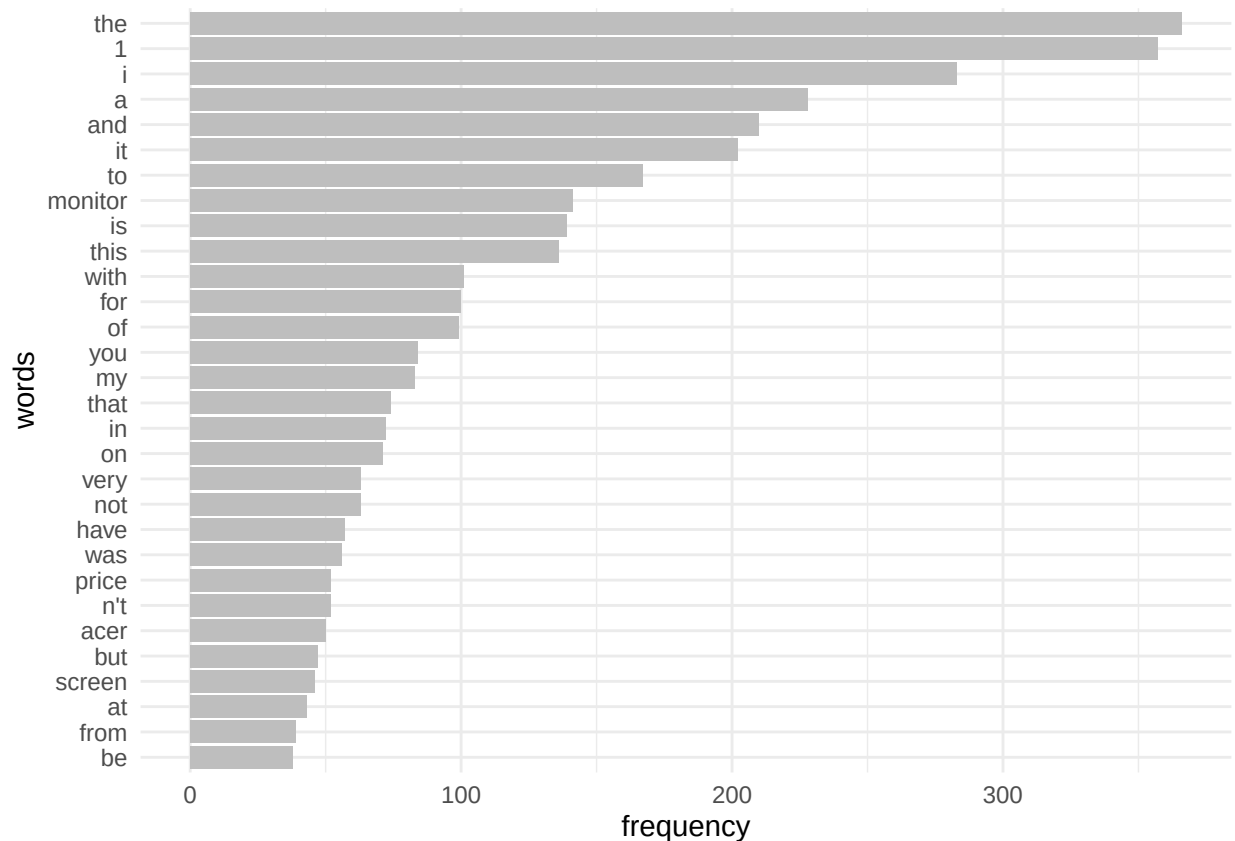
Another way to visualize the term frequencies is using a barplot. For this we need to convert the data to a tidy format (where each word is a different row) and count the number of occurrences of each word.

```
## # A tibble: 6 x 1
##   word
##   <chr>
## 1 i
## 2 purchased
## 3 this
```

```
## 4 monitor
## 5 because
## 6 of
```

```
# unnest_tokens function from tidytext package breaks the text into individual
# tokens (a process called tokenization)
```

```
comp_words |>
  # count the frequency of each word
  count(word) |>
  # arrange the words by its frequency in descending order
  arrange(desc(n)) |>
  # select the top 30 most frequent words
  head(30) |>
  # make a bar plot (reorder words by their frequencies)
  ggplot(aes(x = n, y = reorder(word, n))) +
  geom_col(fill="gray") +
  labs(x = "frequency", y="words") +
  theme_minimal()
```



```
# This selects the most frequent 30 tokens and plot a bar chat
# We see that many of the top words are 'stop words'.
# These stop words need to be removed
```

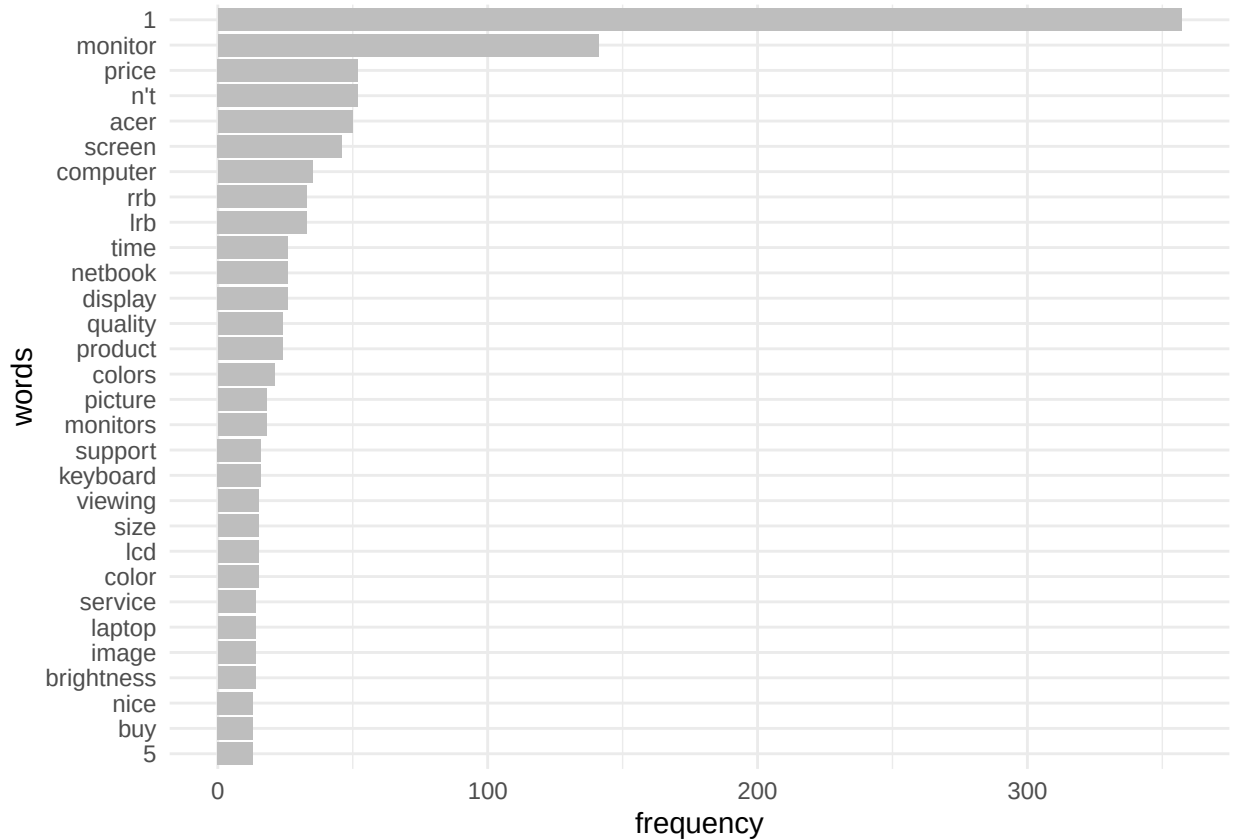
## Remove stop words

```
comp_words_no_stop <-  
  comp_words |>  
  # remove stop words  
  anti_join(stop_words)
```

```
## Joining with 'by = join_by(word)'
```

*# stop\_words is a dataset with stop words. By using anti\_join, you keep only the words in the first dataset that are not in the stop\_words dataset.*

```
comp_words_no_stop |>  
  # count the frequency of each word  
  count(word) |>  
  # arrange the words by its frequency in descending order  
  arrange(desc(n)) |>  
  # select the top 30 most frequent words  
  head(30) |>  
  # make a bar plot (reorder words by their frequencies)  
  ggplot(aes(x = n, y = reorder(word, n))) +  
  geom_col(fill="gray") +  
  labs(x = "frequency", y="words") +  
  theme_minimal()
```

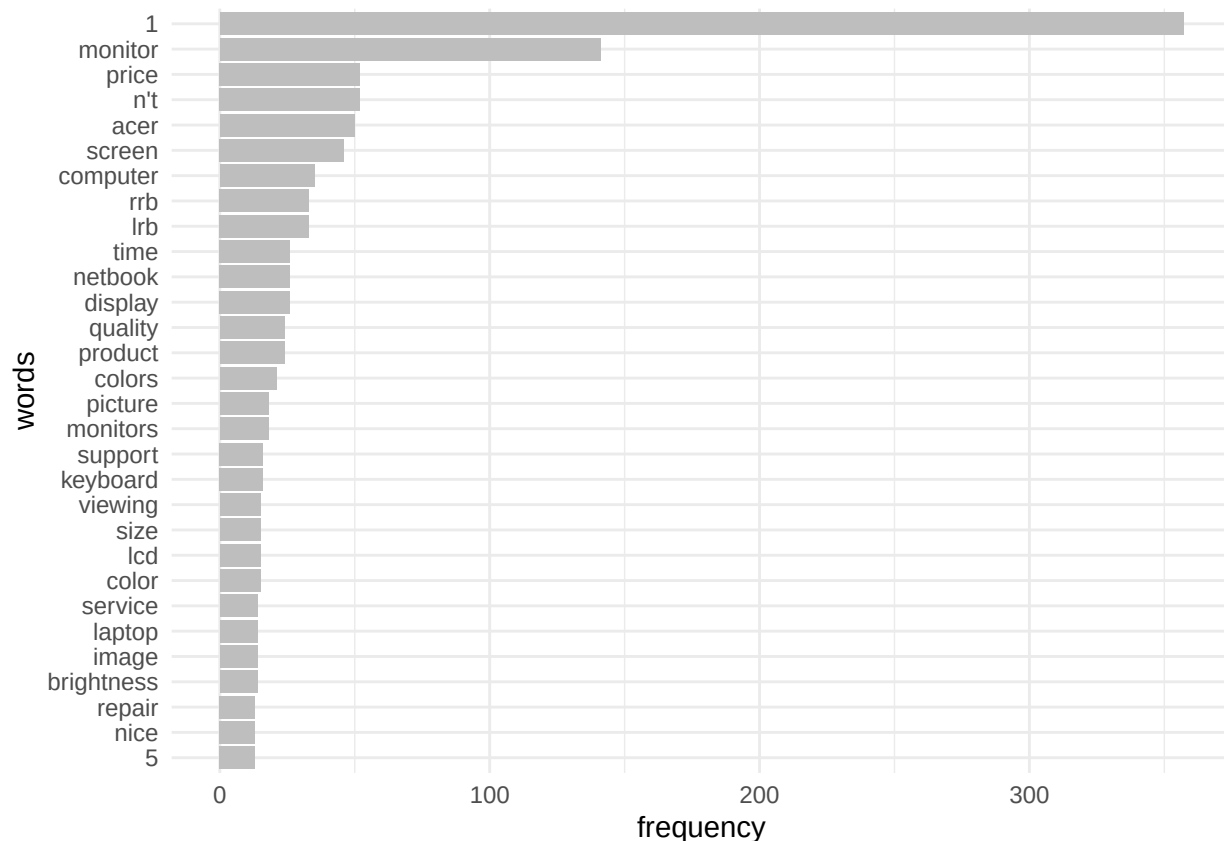


Adjust stopwords list.

```
df_stopwords <-  
  stop_words |>  
  filter(!word %in% c("order", "ordered", "ordering", "orders")) |>  
  rbind(tibble(word = c("buy", "online"), lexicon = c("Me", "Me")))  
  
# remove stop words  
comp_words_no_stop_modified <-  
  comp_words |>  
  anti_join(df_stopwords)
```

## Joining with 'by = join\_by(word)'

```
comp_words_no_stop_modified |>  
  # count the frequency of each word  
  count(word) |>  
  # arrange the words by its frequency in descending order  
  arrange(desc(n)) |>  
  # select the top 30 most frequent words  
  head(30) |>  
  # make a bar plot (reorder words by their frequencies)  
  ggplot(aes(x = n, y = reorder(word, n))) +  
  geom_col(fill="gray") +  
  labs(x = "frequency", y="words") +  
  theme_minimal()
```



*# The resulting barplot looks almost identical to the previous one, just missing  
# "buy," confirming the small custom lexicon barely changed the top-30. Most of  
# the signal was already captured*

## Stemming

When we deal with text, often documents contain different versions of one base word, called a stem. SnowballC is a package based on the famous Porter's stemming algorithm that collapses words into their stem. We shall use the `getStemLanguages` function to find out how many languages the package supports.

```
getStemLanguages()
```

```
## [1] "arabic"      "basque"      "catalan"     "danish"     "dutch"
## [6] "english"     "finnish"     "french"      "german"     "greek"
## [11] "hindi"       "hungarian"   "indonesian"  "irish"      "italian"
## [16] "lithuanian"  "nepali"      "norwegian"   "porter"     "portuguese"
## [21] "romanian"    "russian"     "spanish"     "swedish"    "tamil"
## [26] "turkish"
```

*# supports 26 languages*

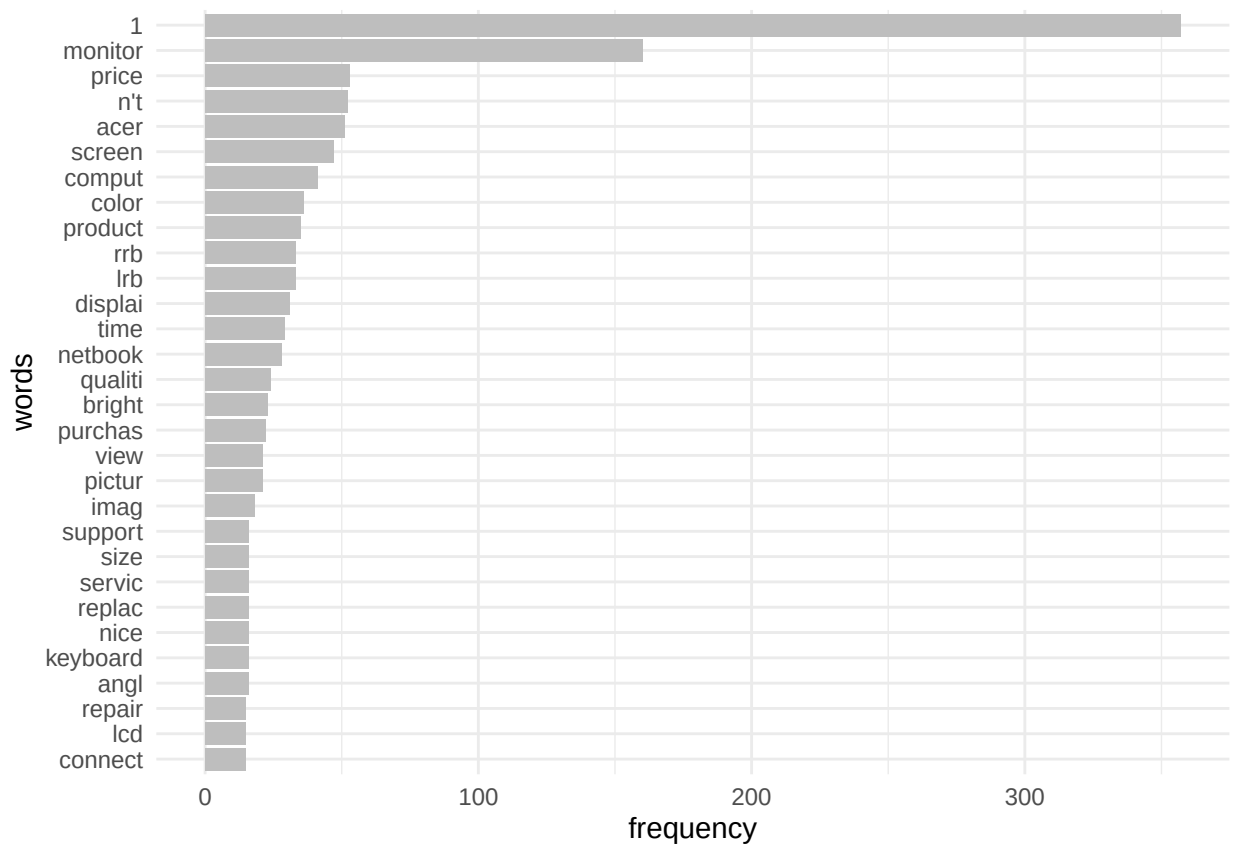
We apply stemming and show the 30 most frequent stemmed words

```

comp_words_stemming <-
  comp_words_no_stop_modified |>
  mutate(stem = wordStem(word))

comp_words_stemming |>
  # count the frequency of each word
  count(stem) |>
  # arrange the words by its frequency in descending order
  arrange(desc(n)) |>
  # select the top 30 most frequent words
  head(30) |>
  # make a bar plot (reorder words by their frequencies)
  ggplot(aes(x = n, y = reorder(stem, n))) +
  geom_col(fill="gray") +
  labs(x = "frequency", y="words") +
  theme_minimal()

```



*# The resulting top terms (monitor, price, acer, screen... etc) are largely the  
 # same as before stemming here, since the top words were already in single-form*

## Regular Expressions

We use regular expressions (regex) to find the reviews in computer\_531, which contain words “Monitor” or “monitor”, “memory” or “Memory”, and “Delivery” or “delivery” and we see how many reviews contain

each pair of words.

```
# reviews containing "Monitor" or "monitor"
reviews_mon <- computer_531 |>
  filter(str_detect(review, "[Mm]onitor"))

# reviews containing "memory" or "Memory"
reviews_mem <- computer_531 |>
  filter(str_detect(review, "[Mm]emory"))

# reviews containing "Delivery" or "delivery"
reviews_del <- computer_531 |>
  filter(str_detect(review, "[Dd]elivery"))

# compare the occurrence of each pair of words
keywords <- c("Monitor", "Memory", "Delivery")
nr_keywords <- c(nrow(reviews_mon),
                 nrow(reviews_mem),
                 nrow(reviews_del))

tibble(keywords, nr_keywords)
```

```
## # A tibble: 3 x 2
##   keywords nr_keywords
##   <chr>      <int>
## 1 Monitor      105
## 2 Memory        5
## 3 Delivery      3
```

```
# We use `str_detect` function from `stringr` package to find the presence of
# patterns of your interest and the `filter` function to only retain the ones
# that contain the patterns.
# Out of the 531 reviews, 105 mention `Monitor`, 5 mention `Memory` and
# 3 mention `Delivery`
# This is inline with the word cloud and frequency plots which suggested that
# monitors are the dominant topic, while memory and delivery are barely discussed.
```

```
## Detect
head(str_detect(computer_531$review,
                 "(\\w+) ([Mm]onitor)"))
```

```
## [1] TRUE TRUE TRUE FALSE TRUE FALSE
```

```
# Returns TRUE/FALSE per review, telling us whether that review contains a word
# immediately followed by "monitor" anywhere in it.
# It is useful for filtering or counting, not for seeing what matched.
```

```
## Extract
head(str_extract(computer_531$review,
                  "(\\w+) ([Mm]onitor)"))
```

```
## [1] "this monitor"      "inch monitor"      "this monitor"
## [4] NA                  "different monitor" NA
```

```
# Pulls out only the substring that matched the whole pattern (both capture groups combined),
# e.g. "this monitor"
# Rows with no match return NA.
# Only the first match per string is returned by default
# str_extract_all would give all matches
```

```
## Subset
head(str_subset(computer_531$review,
                "\\w+ ([Mm]onitor)"))
```

```
## [1] "## I purchased this monitor because of budgetary concerns ."
## [2] "inexpensive[+1][a] ## This item was the most inexpensive 17 inch monitor available to me at the
## [3] "monitor[-1] ## My overall experience with this monitor was very poor ."
## [4] "monitor[-1], picture quality[-1] ## I 've viewed numerous different monitor models since I 'm a
## [5] "## I sent it back and now will search for a better quality monitor ."
## [6] "monitor[-1], colors[-1] ## I used this monitor for 2 years and now it shows a mixture of various
```

```
# Instead of returning TRUE/FALSE or just the matched fragment, this returns the
# entire original review text for every review that matched.
# essentially review[str_detect(review, pattern)].
# It's a shortcut for filtering a character vector down to just the matching elements.
```

```
## Match (Useful with capture groups)
head(str_match(computer_531$review,
                "\\w+ ([Mm]onitor)"))
```

```
##      [,1]      [,2]      [,3]
## [1,] "this monitor" "this"    "monitor"
## [2,] "inch monitor" "inch"    "monitor"
## [3,] "this monitor" "this"    "monitor"
## [4,] NA           NA         NA
## [5,] "different monitor" "different" "monitor"
## [6,] NA           NA         NA
```

```
# This is the most detailed of the four. Column 1 is the full match (same as str_extract),
# and columns 2 and 3 are the two capture groups broken out separately. So we get the
# preceding word and "monitor" as two separate values.
# This is handy when we specifically want to isolate a captured sub-piece
# (e.g. just the word before "monitor") rather than the whole match.
```

```
# use str_extract_all
str_extract_all(computer_531$review[1:10],
                '\\b[A-Z]+\\b',
                simplify = TRUE)
```

```
##      [,1] [,2] [,3]
## [1,] "I"  ""   ""
## [2,] "I"  ""   ""
## [3,] ""   ""   ""
## [4,] ""   ""   ""
## [5,] "I"  "I"  "I"
```

```
## [6,] "A" "I" ""
## [7,] "" "" ""
## [8,] "I" "I" ""
## [9,] "I" "" ""
## [10,] "I" "" ""
```

```
# all matches, returned as a matrix
# Unlike plain str_extract (first match only), str_extract_all returns
# every match per string.
# With simplify = TRUE, instead of returning a list (where each element could have
# a different length), it pads all results into a rectangular matrix: each row is
# one of the first 10 reviews, each column is one match position, and empty
# positions (where a review had fewer all-caps words than the row with the most)
# are filled with "". So a row like "I" "" "" means that review had only one
# all-caps word ("I"), while "A" "I" "" means it had two.
```

```
str_extract(computer_531[[1]][1:20],
             "^.*##")
```

```
## [1] "##" "inexpensive[+1][a] ##"
## [3] "monitor[-1] ##" "screen[-1], picture quality[-1] ##"
## [5] "monitor[-1], picture quality[-1] ##" "screen[-1] ##"
## [7] "Display[-1] ##" "monitor[-1] ##"
## [9] "##" "##"
## [11] "##" "##"
## [13] "##" "monitor[-1], colors[-1] ##"
## [15] "##" "size[+1] ##"
## [17] "computer[+1], quality[+1] ##" "##"
## [19] "keyboard[+1], color[+1] ##" "speed[+1], memory[+1] ##"
```

```
# extracts the characters at the beginning of each line until ## for only
# the first 20 reviews
```

```
computer_531 <- computer_531 |>
  mutate(cleaned_review = str_extract(review, "##.*$"),
         aspect_sentiment = str_extract(review, "^.*##"))
```

```
# check the newly added columns
head(computer_531[-1])
```

```
## # A tibble: 6 x 2
##   cleaned_review                                aspect_sentiment
##   <chr>                                           <chr>
## 1 ## I purchased this monitor because of budgetary concerns . ##
## 2 ## This item was the most inexpensive 17 inch monitor availa~ inexpensive[+1]~
## 3 ## My overall experience with this monitor was very poor .   monitor[-1] ##
## 4 ## When the screen was n't contracting or glitching the over~ screen[-1], pic~
## 5 ## I 've viewed numerous different monitor models since I 'm~ monitor[-1], pi~
## 6 ## A week out of the box and I began to see slight contracti~ screen[-1] ##
```

```
# new column added to the computer_531 data frame with the name cleaned_review,
# which contains only the review text. And another column added with the
# name aspect_sentiment, which contains the aspects and sentiment words.
```

For an even cleaner version you can use regex's lookbehind and lookahead features to drop the ## parts

```
computer_531 <- computer_531 |>
  mutate(cleaned_review = str_extract(review,"(?<=##).*$"),
         aspect_sentiment = str_extract(review,"^.*(=?=##)"))

# check the newly added columns
head(computer_531[-1])
```

```
## # A tibble: 6 x 2
##   cleaned_review                                aspect_sentiment
##   <chr>                                           <chr>
## 1 " I purchased this monitor because of budgetary concerns ." ""
## 2 " This item was the most inexpensive 17 inch monitor availab~ "inexpensive[+1~
## 3 " My overall experience with this monitor was very poor ." "monitor[-1] "
## 4 " When the screen was n't contracting or glitching the overa~ "screen[-1], pi~
## 5 " I 've viewed numerous different monitor models since I 'm ~ "monitor[-1], p~
## 6 " A week out of the box and I began to see slight contractio~ "screen[-1] "
```

```
# No leftover `##` characters cluttering either column
```

```
# define sum_list function which adds the scores
sum_list <- function(list_values) {
  sum(as.numeric(list_values))
}

computer_531 <- computer_531 |>
  # sentiment_score: extract the scores and sum them using the sum_list function
  mutate(sentiment_score = map_dbl(str_extract_all(aspect_sentiment, "[-|\\+|\\d+)", sum_list),
    # assign negative when score < 0
    sentiment = case_when(sentiment_score < 0 ~ "negative",
    # assign negative when score > 0
    sentiment_score > 0 ~ "positive",
    # assign neutral otherwise
    TRUE ~ "neutral"))

head(computer_531)
```

```
## # A tibble: 6 x 5
##   review                                cleaned_review aspect_sentiment sentiment_score sentiment
##   <chr>                                <chr>           <chr>                <dbl> <chr>
## 1 ## I purchased this~ " I purchased~ ""                0 neutral
## 2 inexpensive[+1][a] ~ " This item w~ "inexpensive[+1~ 1 positive
## 3 monitor[-1] ## My o~ " My overall ~ "monitor[-1] " -1 negative
## 4 screen[-1], picture~ " When the sc~ "screen[-1], pi~ -2 negative
## 5 monitor[-1], pictur~ " I 've viewe~ "monitor[-1], p~ -2 negative
## 6 screen[-1] ## A wee~ " A week out ~ "screen[-1] " -1 negative
```

```
# This create a new column sentiment with the values positive, negative and neutral.  
# The code assigns neutral in case when there is no aspect in the corresponding  
# column or the sum of scores is equal to zero, and it assigns negative or positive  
# when the sum score is lower or higher) than zero repectively.
```

```
computer_531 |>  
  arrange(sentiment_score) |>  
  tail(1) |>  
  pull(cleaned_review) |>  
  strwrap() |>  
  writeLines()
```

```
## After much delay I received my Acer Netbook Aspire One and I can teell  
## you I am very satisfied with this computer , is very pretty , well  
## finished , high quality , so quiet , not heated than other netbook or  
## at least I think it is within in normal ranges , the keyboard is very  
## smooth and very well laid out , the screen functions are very practical  
## in a nutshell is an excellent buy and I am very pleased with the  
## purchase , I recommend it .
```

```
# arrange(sentiment_score) sorts the whole table by score, ascending (most negative  
# first, most positive last)  
  
#tail(1) grabs the last row - i.e., the single review with the highest sentiment score.  
  
# pull(cleaned_review) extracts just the cleaned_review text column from that  
#row (rather than the whole row/data frame).  
  
# The result is the actual text of the most positive review in the dataset
```